

SECTION B - PRACTICAL PROBLEMS

Problem 1. DNA Sequence Analysis Suite · Difficulty: Medium

Context: A genomics lab receives raw DNA sequences from sequencing machines. You must build a validated preprocessing and analysis library before the data can be passed to ML models.

Tasks:

1. Write `gc_content(dna: str) → float`: returns the percentage of G+C bases in the sequence (ignore case, skip non-ATCG characters).
2. Write `reverse_complement(dna: str) → str`: returns the reverse complement (A↔T, G↔C) of the given sequence.
3. Write `naive_exact_match(text: str, pattern: str) → list[int]`: returns all starting positions (0-indexed) where pattern occurs in text.
4. Write `nucleotide_freq(dna: str) → dict`: returns a dict {"A":n, "T":n, "G":n, "C":n} with counts.
5. Integrate all four into a pipeline function `analyze_sequence(dna)` that returns a summary dict with `gc_percent`, `at_percent`, `most_frequent_nucleotide`, and `reverse_complement`.

Constraints: No external libraries. Handle empty strings and invalid characters gracefully.

Expected Output Hint: "ATCGATCGATCG" → `gc_content=50.0`, `most_frequent=A or T (tied)`, `rc="CGATCGATCGAT"`

Problem 2. Student Grade Analyzer · Difficulty: Medium

Context: A university registrar needs a script to process semester results for a cohort of 20+ students.

Tasks:

1. Store a list of dicts with keys: `name (str)`, `scores (list of 5 subject marks out of 100)`.
2. Compute each student's average and assign letter grades: A (≥85), B (75-84), C (60-74), D (45-59), F (<45).
3. Find the top-3 and bottom-3 students by average - without sorting the original list.
4. Build a `grade_distribution` dict: {"A":n, "B":n, ...} using a dict comprehension.
5. Identify subjects (by index 0-4) where the class average is below 60 - these are flagged for review.

Constraints: No numpy or statistics module. Compute mean manually.

Expected Output Hint: Output a formatted summary table with `student`, `average`, `grade`, and a `flagged subject list`.

Problem 3. Word Frequency Counter · Difficulty: Medium

Context: Text analytics preprocessing - a fundamental step in every NLP pipeline before vectorisation.

Tasks:

1. Write `word_freq(text: str, top_n: int = 10) → list[tuple]`: returns top_n (word, count) tuples sorted by count descending.
2. Case-insensitive, strip punctuation (.,!?:;), split on whitespace.
3. Exclude stop words: {"the", "a", "an", "is", "in", "of", "and", "to", "that", "it", "for", "on", "with"}.
4. Count WITHOUT using `collections.Counter` - use a plain dict and `.get()`.
5. Return a formatted string report: "Rank 1: data (12 times)", etc.

Constraints: No imports allowed (no `Counter`, `re`, or `string` module).

Expected Output Hint: On a paragraph about machine learning, top words should be ML-domain terms.

Problem 4. Log File Parser with Regex · Difficulty: High

Context: A bioinformatics pipeline emits structured log files. You need to parse and summarise them for QA reports.

Tasks:

1. Write `parse_pipeline_log(log_text: str) → dict` with keys: `total_lines`, `by_level` (INFO/WARN/ERROR counts), `errors` (list of dicts), `warnings` (list of dicts), `duration_seconds` (float).
2. Each line format: YYYY-MM-DD HH:MM:SS LEVEL MODULE message text.
3. Use `re.match()` or `re.findall()` to extract timestamp, level, module, and message.
4. Compute duration as time difference between last and first log entry.
5. errors and warnings lists: each item is {time: str, module: str, message: str}.

Constraints: Must use the `re` module. `datetime` module allowed for time parsing.

Expected Output Hint: On the provided sample log, `by_level={INFO:4, WARN:2, ERROR:1}`, `duration=~120` seconds.

Problem 5. Caesar Cipher Toolkit · Difficulty: Low–Medium

Context: A simple encryption task commonly used in technical screening interviews at software companies.

Tasks:

1. Write `caesar_encrypt(text: str, shift: int) → str`: shifts each letter by shift positions, wraps around (z+1=a), preserves case, leaves non-alpha characters unchanged.
2. Write `caesar_decrypt(text: str, shift: int) → str`: reverses the encryption.
3. Write `brute_force_crack(ciphertext: str) → list[str]`: returns all 25 possible decryptions (one per shift).

4. Write `crack_by_frequency(ciphertext: str) → tuple[str, int]`: identifies the most likely plaintext by comparing letter frequency to English ("e" most common). Returns (plaintext, shift).

Constraints: String operations only. No external libraries.

Expected Output Hint: `caesar_encrypt("Hello!", 3) → "Khoor!"`. Brute-force should reveal the shift when given known ciphertext.

Problem 6. Run-Length Encoding (RLE) · Difficulty: Medium

Context: RLE is used in bioinformatics (sequence compression) and is a standard coding interview question at data-focused companies.

Tasks:

1. Write `rle_encode(s: str) → str`: "AAABBC" → "3A2B1C".
2. Write `rle_decode(s: str) → str`: "3A2B1C" → "AAABBC".
3. Handle edge cases: single characters, mixed case, empty string.
4. Write `rle_ratio(s: str) → float`: returns the compression ratio `len(encoded)/len(original)`.
5. Apply to a DNA sequence "AAATTTCCCGGGAAATTT" and report whether encoding saves space.

Constraints: No imports. Loop-based implementation required (no `itertools.groupby`).

Expected Output Hint: `rle_encode("AAABBC")="3A2B1C"`. `Ratio<1.0` means compression saved space.

Problem 7. Inventory Management System · Difficulty: Medium

Context: A common interview exercise that tests dict manipulation, function design, and error handling.

Tasks:

1. Write a class-free inventory system using a single dict as data store: `inventory = {product_id: {name, qty, price}}`.
2. Implement `add_product(inv, pid, name, qty, price)`, `remove_product(inv, pid)`, `update_qty(inv, pid, delta)` (delta can be negative for sales).
3. Implement `get_total_value(inv) → float`: sum of `qty*price` across all products.
4. Implement `low_stock_alert(inv, threshold=10) → list`: returns list of product names where `qty < threshold`.
5. Implement `search_by_name(inv, query: str) → list`: case-insensitive partial match on product name.

Constraints: No classes - pure function + dict approach. Raise `ValueError` for invalid operations (e.g., removing from empty).

Expected Output Hint: After adding 5 products and selling 8 units of product 1, `get_total_value()` and `low_stock_alert()` should reflect the updated state.

Problem 8. Grade Distribution Histogram (Text-Based) · Difficulty: Low

Context: *Visualising data distributions without libraries is a common first analytics task.*

Tasks:

1. Given a list of 30 student scores (0-100), group them into buckets: 0-9, 10-19, ..., 90-100.
2. Build the frequency dict using a loop and integer division (`score//10`).
3. Print a horizontal bar chart using `*` characters: each `*` represents one student.
4. Print summary stats below: min, max, mean (manual), median (manual sort + middle index).
5. Highlight the most populous bucket with a different marker (e.g., `#`).

Constraints: No matplotlib, numpy, or statistics. Pure Python.

Expected Output Hint: 0-9 : ** | 10-19: *** | ... | 90-100: ##### (highlighted) with stats below.

Problem 9. Matrix Operations Without NumPy · Difficulty: Medium

Context: *Understanding list-of-lists as matrices prepares students for the transition to NumPy arrays.*

Tasks:

1. Write `matrix_add(A, B)` and `matrix_multiply(A, B)` for 2D list-of-lists. Raise `ValueError` if dimensions are incompatible.
2. Write `transpose(M)` → 2D list: rows become columns.
3. Write `row_sums(M)` and `col_sums(M)`: return list of sums for each row/column.
4. Write `identity(n)` → $n \times n$ identity matrix as list-of-lists.
5. Verify: for a 3×3 matrix `M`, `M × identity(3) == M`.

Constraints: No numpy. Nested list comprehensions encouraged.

Expected Output Hint: `matrix_multiply([[1,2],[3,4]], [[5,6],[7,8]]) = [[19,22],[43,50]]`.

Problem 10. Reusable Analytics Toolkit (Module Design) · Difficulty: High

Context: *A production-quality analytics utility module - this mirrors take-home assignments at data analyst roles.*

Tasks:

1. Write `descriptive_summary(data: list)` → dict with keys: `mean`, `median`, `mode`, `std`, `iq_range`, `skewness_direction` ("left", "symmetric", "right" based on mean vs median).
2. Write `normalize(data: list, method="minmax")` → list: supports "minmax" (0-1 range) and "zscore" (subtract mean, divide std).
3. Write `correlate(x: list, y: list)` → float: Pearson r coefficient without numpy.

4. Write `outlier_flags(data: list, method="iqr") → list[bool]`: True where value is an outlier (IQR or Z-score method).

5. Write a `cli_report(data: list)` function that prints a formatted text summary using all functions above.

Constraints: No numpy, statistics, or scipy. All math from scratch. Include assert-based unit tests for each function.

Expected Output Hint: On `[1,2,2,3,100]`: `mean=21.6, median=2, mode=2, outlier_flags=[F,F,F,F,T], skewness="right"`.

Problem 11. Set Operations on Gene Lists · Difficulty: Low

Context: *Comparative genomics requires identifying shared and unique genes across multiple experimental groups.*

Tasks:

1. Given gene lists for three patient groups (A, B, C) as Python sets, find: genes present in ALL three groups (core genes).
2. Find genes unique to each group (not in either of the other two).
3. Find genes shared by exactly two groups - compute for all three pair combinations.
4. Compute and print a Venn-diagram summary: |A only|, |B only|, |C only|, |A∩B only|, |A∩C only|, |B∩C only|, |A∩B∩C|.
5. Write it as a function `venn_summary(A, B, C) → dict` with descriptive keys.

Constraints: Use only set operations (`|`, `&`, `-`, `^`). No loops.

Expected Output Hint: Given sample gene lists, produce all 7 Venn regions as integer counts.

Problem 12. Phone Number Normaliser with Regex · Difficulty: Medium

Context: *Data cleaning of contact information is among the most frequent tasks in a data engineering pipeline.*

Tasks:

1. Write `normalise_phone(number: str) → str`: accepts formats like +92-321-1234567, 0321-1234567, 03211234567, (0321) 123 4567, and returns a canonical form: +92-3XX-XXXXXXX.
2. Write `validate_phone(number: str) → bool`: True if the normalised number is a valid Pakistani mobile format.
3. Write `extract_phones(text: str) → list[str]`: finds all phone numbers in a paragraph of text.
4. Handle international numbers with country code +1, +44, +92.
5. Return a summary dict: `{valid: [...], invalid: [...], normalised: [...]}` from a batch list.

Constraints: Must use the `re` module. Regex patterns must be explained in comments.

Expected Output Hint: ["0321-1234567", "+92 321 1234567", "03211234567"] → all normalise to "+92-321-1234567".

Problem 13. Codon Translator (DNA → Protein) · Difficulty: High

Context: A capstone bioinformatics task combining strings, dicts, loops, and functions - represents real ML preprocessing work.

Tasks:

1. Implement `translate_dna(sequence: str) → str`: reads DNA in triplets (codons), returns the amino acid sequence using the provided `CODON_TABLE` dict.
2. Stop translation at the first stop codon (TAA, TAG, TGA) - do not include the stop in output.
3. Raise `ValueError` if the sequence contains bases other than ATCG (case-insensitive).
4. Handle sequences not divisible by 3: translate the complete codons only and append a warning.
5. Write `find_orfs(dna: str) → list[str]`: finds all Open Reading Frames (start ATG to stop codon) and returns each as a translatable subsequence.

Constraints: Provide the starter `CODON_TABLE` dict (standard genetic code). No Biopython.

Expected Output Hint: `translate_dna("ATGAAATTTGCATAG") → "MKF"` (Met-Lys-Phe, then stop).

Problem 14. Data Cleaning Pipeline · Difficulty: Low–Medium

Context: Raw survey data from a university department contains multiple quality issues. Your job is to standardise it before analysis.

Tasks:

1. Load a list of student dicts (provided as hard-coded Python list) with messy fields: mixed-case names ("john doe"), age as string ("22 years"), CGPA as string ("3.75/4"), gender as M/Male/male.
2. Write `clean_record(record: dict) → dict`: normalise name (title case, stripped), extract integer age, extract float CGPA, map gender to "M"/"F".
3. Write `validate_record(record: dict) → tuple[bool, list]`: returns (is_valid, [error_messages]) for range checks (age 15-60, CGPA 0-4).
4. Produce a cleaning report: `n_total`, `n_valid`, `n_cleaned`, list of issues per record.
5. Output a cleaned list of dicts and a summary table.

Constraints: No pandas. Pure Python string methods, regex for extraction is allowed.

Expected Output Hint: Input: `{name: "JOHN DOE ", age: "22 years", cgpa: "3.75/4.0", gender: "Male"}` → `{name: "John Doe", age: 22, cgpa: 3.75, gender: "M"}`.

Problem 15. Capstone - Complete Analytics Pipeline (Pre-Assessment) · Difficulty: High

Context: *This problem mirrors the format of the module practical exam. It integrates all six class topics into a single coherent task. Time yourself: target completion in 45 minutes.*

Tasks:

1. DATASET: Use the provided hard-coded list of 50 employee dicts: {id, name, department, salary, years_exp, performance_score (1-5)}.
2. CLEANING (strings/types): Normalise names, ensure salary and score are numeric, flag records with missing/invalid data.
3. ANALYSIS (dicts/lists): Compute mean salary by department using a dict. Find top-5 performers. Find employees with salary below the departmental average.
4. CONTROL FLOW: Classify each employee as "underpaid", "fairly_paid", or "overpaid" relative to department mean $\pm$ 1 standard deviation.
5. FUNCTIONS: Wrap the entire pipeline in analyse_workforce(data: list) $\rightarrow$ dict with keys: clean_count, dept_stats, top_performers, classification_counts, anomalies.

Constraints: No pandas, numpy, or statistics. All calculations from scratch. Write 3 assert-based tests at the bottom to verify your pipeline.

Expected Output Hint: classify_counts should have all three categories populated. Top-5 should have performance_score=5 or 4.